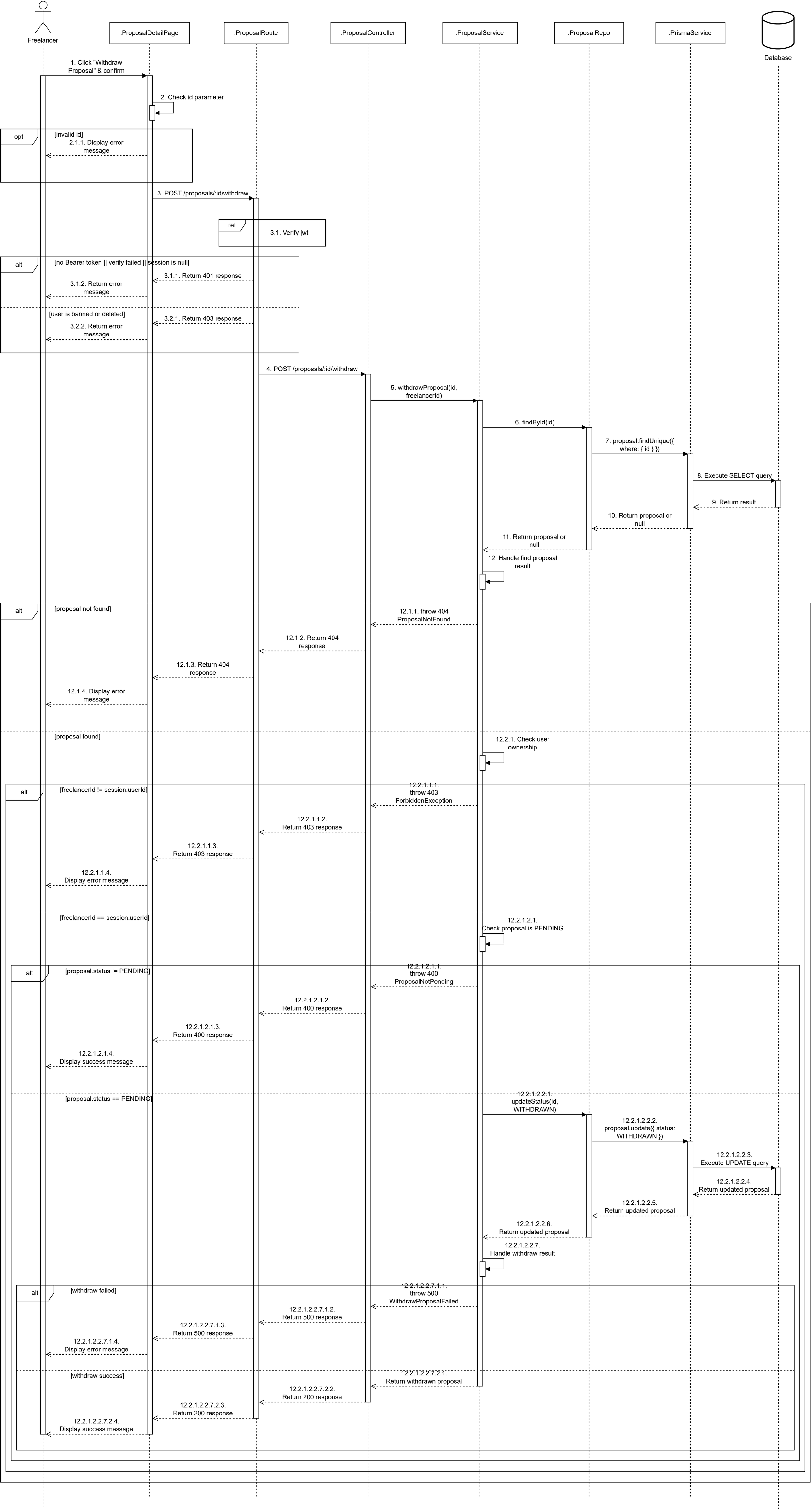

Freelancer
:ProposalDetailPage
:ProposalRoute
:ProposalController
:ProposalService
:ProposalRepo
:PrismaService
Database
1. Click "Withdraw Proposal" & confirm
2. Check id parameter
opt
[invalid id]
2.1.1. Display error message
3. POST /proposals/:id/withdraw
ref
3.1. Verify jwt
alt
[no Bearer token || verify failed || session is null]
3.1.1. Return 401 response
3.1.2. Return error message
[user is banned or deleted]
3.2.1. Return 403 response
3.2.2. Return error message
4. POST /proposals/:id/withdraw
5. withdrawProposal(id, freelancerId)
6. findById(id)
7. proposal.findUnique({ where: { id } })
8. Execute SELECT query
9. Return result
10. Return proposal or null
11. Return proposal or null
12. Handle find proposal result
alt
[proposal not found]
12.1.1. throw 404 ProposalNotFound
12.1.2. Return 404 response
12.1.3. Return 404 response
12.1.4. Display error message
[proposal found]
12.2.1. Check user ownership
alt
[freelancerId != session.userId]
12.2.1.1.1. throw 403 ForbiddenException
12.2.1.1.2. Return 403 response
12.2.1.1.3. Return 403 response
12.2.1.1.4. Display error message
[freelancerId == session.userId]
12.2.1.2.1. Check proposal is PENDING
alt
[proposal.status != PENDING]
12.2.1.2.1.1. throw 400 ProposalNotPending
12.2.1.2.1.2. Return 400 response
12.2.1.2.1.3. Return 400 response
12.2.1.2.1.4. Display success message
[proposal.status == PENDING]
12.2.1.2.2.1. updateStatus(id, WITHDRAWN)
12.2.1.2.2.2. proposal.update({ status: WITHDRAWN })
12.2.1.2.2.3. Execute UPDATE query
12.2.1.2.2.4. Return updated proposal
12.2.1.2.2.5. Return updated proposal
12.2.1.2.2.6. Return updated proposal
12.2.1.2.2.7. Handle withdraw result
alt
[withdraw failed]
12.2.1.2.2.7.1.1. throw 500 WithdrawProposalFailed
12.2.1.2.2.7.1.2. Return 500 response
12.2.1.2.2.7.1.3. Return 500 response
12.2.1.2.2.7.1.4. Display error message
[withdraw success]
12.2.1.2.2.7.2.1. Return withdrawn proposal
12.2.1.2.2.7.2.2. Return 200 response
12.2.1.2.2.7.2.3. Return 200 response
12.2.1.2.2.7.2.4. Display success message